How Many Training Notes? Sample Efficiency of Linear, Nearest-Neighbour, Tree and Neural Classifiers for Forged-Banknote Detection

Maanas **Advit**
University of Pennsylvania

Abstract

On the UCI Banknote Authentication benchmark, a tuned logistic regression already misclassifies only 0.56% of notes in cross-validation, so ranking methods by final accuracy compares noise. We instead ask how many labelled training notes each method needs. We compare logistic regression (the baseline) and k-nearest neighbours, with random forests, gradient boosting and a small neural network as extensions, along learning curves from 10 to 800 training notes. Each size has 50 paired draws, and every method is tuned by cross-validation inside each draw. Logistic regression learns fastest of the core methods at first (2% error from about 71 notes) but levels off near 0.6%. kNN overtakes it at about 100 notes and reaches 1% error from about 149 notes, against 345. Tree ensembles are the slowest learners, while an 8-unit neural network has the lowest error at every size and reaches 1% from about 92 notes. A check for several images of the same physical specimen leaves these results unchanged. On the 270-note test set, all methods make 0–3 errors and are statistically tied, which is why the learning curves, not the test set, carry the comparison.

1 Introduction

Counterfeit detection is a natural use for a cheap classifier. A scanner photographs a note, a few image statistics are computed, and a model flags it as genuine or forged. The UCI Banknote Authentication data [1] is a standard small benchmark for this task, with four wavelet features per image. On it, standard methods are almost perfect: a tuned logistic regression misclassifies 0.56% of notes in 5-fold cross-validation (CV). Ranking methods by their final accuracy here would mostly compare noise, because with a few hundred held-out notes the difference between 0.3% and 0.6% error is one or two notes.

We therefore ask a question that still separates methods when accuracy saturates: *how many labelled training notes does each method need to reach a given error rate?* In practice, labelled forgeries are the expensive part of building such a system, because each one must be produced or seized and then verified. A method that reaches 1% error from 100 notes is more useful than one that needs 800, even if both are perfect at 1,000.

We compare two methods taught early in the course, logistic regression (the baseline) and k-nearest neighbours (kNN), and three later ones as an extension: random forests, gradient boosting and a one-hidden-layer neural network. We draw learning curves from 10 to 800 training notes with 50 random draws per size, tune every method by CV inside each draw, and compare methods on the same draws. Our contributions are:

ESE 5420 course project (University of Pennsylvania, Fall 2026), written in the format of a machine learning paper. Not peer reviewed.

- a leakage-aware protocol for this dataset: duplicate removal, scaling and tuning inside each training set, an untouched test set, and a check for several images of the same physical specimen;
- paired learning curves with standard errors, and the number of notes each method needs to reach 2% and 1% error, with bootstrap intervals;
- an honest reading of where the differences are real and where they are ties, including the one-time test-set result.

Among the two core methods we find a crossover. Logistic regression learns fastest at first, reaching 2% error from about 71 notes, but levels off near 0.6% because no linear boundary separates the classes. kNN starts worse, catches up at about 100 notes and reaches 1% from about 149 notes, against 345 for logistic regression. Among the extensions, the tree ensembles are the least sample-efficient methods we tried, while a small neural network with 8 hidden units has the lowest mean error at every size from 10 notes up and reaches 1% from about 92 notes. On the 270 test notes, all five methods make between 0 and 3 errors, so the test set cannot separate them; the learning curves can.

2 Related work

Dataset. The data come from the authentication work of Gillich and Lohweg [2]: images of genuine and forged banknote-like specimens were taken with an industrial camera and summarised by statistics of a wavelet transform. Lohweg released the four features on the UCI repository [1]. Because most standard methods score close to 100% on it, a single accuracy figure says little about which method to prefer.

Learning curves for comparing classifiers. Perlich, Provost and Simonoff [3] compared logistic regression and tree induction across training sizes in a large-scale study of many datasets. They found that logistic regression tends to win on small samples and trees on large ones, so that the curves can cross and a comparison at one size can mislead. Ng and Jordan [4] showed a related effect between generative and discriminative models: the method with the lower asymptotic error is not always the faster learner. Viering and Loog [5] review the shapes learning curves take, including non-monotone ones. Our study applies this learning-curve view to a dataset where final accuracy is saturated.

Fair evaluation. Choosing hyperparameters and estimating error on the same folds biases the estimate downwards [6, 7], so we use nested CV for the baseline and tune inside every training draw. Leakage between training and evaluation data, for example near-copies of the same object, can make a model look better than it is [8]. To compare two classifiers on one test set we use McNemar's test, as recommended by Dietterich [9].

Methods. We use standard forms of logistic regression and CV [10], nearest-neighbour classification [11], random forests [12], gradient boosting [13] and a small multilayer perceptron, all in scikit-learn [14].

3 Data

The UCI file [1] (CC BY 4.0) has 1,372 rows. Each has four inputs, the variance, skewness, kurtosis and entropy of a wavelet-transformed image, and a binary class. There are no missing values. We remove 24 exact duplicate rows, all from class 0, which leaves 1,348 notes, 45.3% of them in class 1 (Table 1). UCI does not say which class is forged, and secondary sources disagree, so we call class 1 "forged". Error and AUC do not depend on this choice; missed forgeries and false alarms would swap.

We make one stratified 80/20 split with a fixed random seed[1] into 1,078 development notes and 270 test notes, and store it with the data. All tuning, learning curves and model choices use the

[1]The seed is 5420, the course number. It is only a fixed starting point for the random-number generator, so that anyone rerunning the code gets the same split; it was not chosen by looking at results. Every later random choice derives its seed from it.

Table 1: Data size and shape at each stage. "Forged" is the share of class 1.

Stage	Rows	Inputs	Forged
Raw UCI file	1372	4	44.5%
After removing exact duplicates	1348	4	45.3%
Development set (80%)	1078	4	45.3%
Test set (20%, held out)	270	4	45.2%

development notes only. The test notes were used once, after every modelling choice was fixed (Section 5.4).

Several images per specimen? The file has no specimen identifiers, but the class-1 rows are strongly correlated in file order: the lag-1 autocorrelation of skewness is 0.62 for class 1 and about 0 for class 0, with a repeating pattern roughly every 8 rows. This suggests that the forged class contains several images of each physical specimen in a row. If near-identical images of one note fall on both sides of a split, a method that memorises training points, such as 1-nearest-neighbour, gains an unfair advantage. We cannot remove this risk without identifiers, so we test for it: we treat each run of 8 consecutive rows of one class as a pseudo-specimen and repeat the key experiments with whole pseudo-specimens kept on one side of every split (Section 5.3).

4 Method

Classifiers. All inputs are standardised inside each training set (a scikit-learn pipeline [14]), so no statistics leak from evaluation rows. Each method has a small grid tuned by stratified 5-fold CV on the training set alone:

- **Logistic regression** (baseline): $C \in \{10^{-3}, 10^{-2.5}, \ldots, 10^{3}\}$, 13 values.
- k**NN**: $k \in \{1, 3, 5, 7, 9, 11, 15, 21, 31\}$, limited to k smaller than an inner training fold.
- **Random forest**†: 200 trees; features per split $m \in \{1, 2, 4\}$, minimum leaf size $\in \{1, 3\}$.
- **Gradient boosting**†: 200 trees; learning rate $\eta \in \{0.05, 0.2\}$, depth $\in \{1, 3\}$.
- **Neural network**†: one hidden layer with 8 or 32 ReLU units, weight decay $\alpha \in \{10^{-4}, 10^{-2}, 1\}$, trained by L-BFGS.

†Later-course extensions (ensembles and neural networks). No baseline number depends on them. The grids are deliberately small so that every method can be tuned inside each of 2,750 training draws. Appendix A.4 shows that a 64-setting forest grid on all development data does not do better than the small one.

Baseline estimate. On all 1,078 development notes we report the 5-fold CV error of logistic regression at the chosen C, with the standard error (SE) of the five fold errors, SD/$\sqrt{5}$. Because C is chosen on the same folds, we also report nested 5×5 CV.

Learning curves. For each training size $n \in \{10, 20, 30, 50, 100, 150, 200, 300, 400, 600, 800\}$ and each of 50 draws, we sample n development notes stratified by class, tune each method by 5-fold CV inside the sample, and score it on all remaining development notes (between 278 and 1,068). All methods see the same 50 training sets at each size, so we compare them with paired differences: for method j and draw d, $\Delta_{jd} = \text{err}_{jd} - \text{err}_{\text{LR},d}$, reported as the mean over draws ± its SE. We call a difference clear when it exceeds 2 SE.

Notes needed to reach a target. For each method we find where the mean curve first falls to 2% and to 1% error, by linear interpolation between neighbouring sizes on log–log axes. We get a 95% interval by resampling the 50 draws at every size with replacement (2,000 bootstrap replicates) and repeating the calculation.

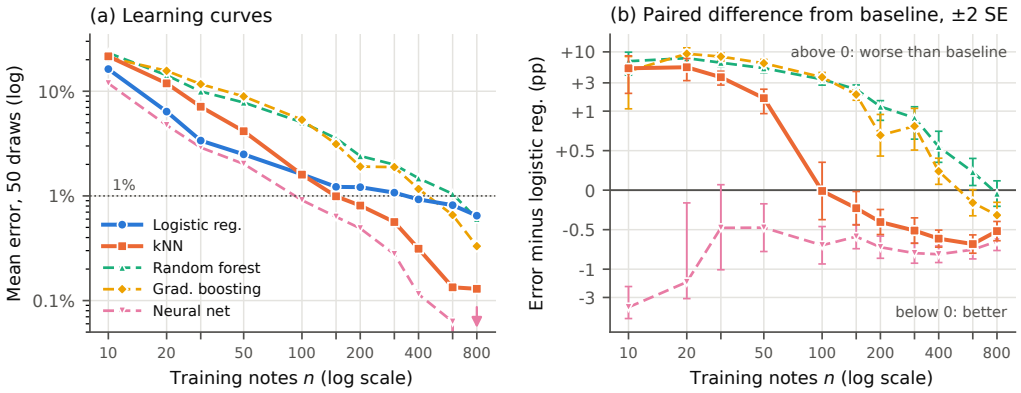


Figure 1: (a) Mean error on held-out development notes over 50 draws per size (log–log). Solid lines are the core methods, dashed lines the later-course extensions. The arrow marks the neural network at $n = 800$, where all 50 draws had zero errors. (b) Paired difference from logistic regression on the same draws, in percentage points, with ± 2 SE bars (symmetric-log axis, linear between -1 and $+1$). Below 0 is better than the baseline.

Test evaluation. Each method is tuned by 5-fold CV on all development notes, refitted on them, and scored once on the 270 test notes. We give Wilson 95% intervals for each error, and compare each method with the baseline by an exact McNemar test on the notes where the two disagree [9].

5 Experiments and results

5.1 Baseline

On all 1,078 development notes, logistic regression chooses $C = 100$, almost no regularisation, and reaches a 5-fold CV error of 0.56% $\pm$ 0.09%: 6 wrong notes. Nested CV, which also accounts for choosing C, gives 0.74% $\pm$ 0.32% (Appendix A.1). The majority-class rule errs on 45.3%. Even without regularisation, logistic regression still gets 4 of its own training notes wrong, so the classes are not linearly separable in these features. The baseline's remaining error is therefore bias, not a lack of data, and that already predicts how its learning curve will look.

5.2 Learning curves

Figure 1 shows the main result; full numbers are in Appendix A.2.

Logistic regression vs. kNN. Logistic regression is clearly better at small sizes: at 50 notes it errs on 2.50% against 4.15% for kNN, a paired gap of +1.66 $\pm$ 0.34 points. The two tie at 100 notes (1.61% vs. 1.60%). From 150 notes kNN is clearly better (-0.23 $\pm$ 0.11 points) and the gap keeps growing, because logistic regression flattens out while kNN does not: at 800 notes the errors are 0.65% and 0.13%, and kNN wins 41 of 50 draws. Inner CV picks $k = 1$ in most draws at every size (Table 8). This matches the crossing curves Perlich et al. [3] report for logistic regression against a more flexible learner, which has the lower error at large sizes but needs more data to get there.

Extensions. The two tree ensembles are worse than the baseline at every size up to 400 notes, often by several points: at 100 notes the forest errs on 5.05% and boosting on 5.34%. Boosting is clearly better than the baseline only at 800 notes (0.33%), and the forest only ties it there. A larger forest grid on all development notes does not help (Appendix A.4), so this is not a tuning problem. The small neural network, in contrast, has the lowest mean error of all five methods at every size. At 10 notes it errs on 11.90% against 16.28% for the baseline (-4.39 $\pm$ 1.21 points), and at 800 notes none of its 50 draws makes a single error. CV most often picks the 8-unit network with weak weight decay.

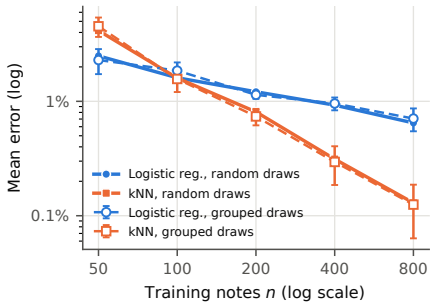


Figure 2: Random draws (solid) vs. draws that keep each 8-row pseudo-specimen on one side of every split (dashed, ± 2 SE).

Table 2: Training notes needed for the mean error to reach 2% and 1%, interpolated on the learning curves, with bootstrap 95% intervals over draws. †Extension.

Method	2% error		1% error	
	n	95% CI	n	95% CI
Logistic regression	71	58–83	345	278–415
kNN	85	77–95	149	131–176
Random forest†	297	247–326	614	560–648
Gradient boosting†	195	184–304	446	403–479
Neural network†	50	41–58	92	80–108

Table 3: One-time evaluation on the 270 test notes (122 forged), after tuning by 5-fold CV on all development notes. "Missed" are forged notes called genuine, "False al." genuine notes called forged. The interval is a Wilson 95% interval; p is an exact McNemar test against logistic regression. †Extension.

Method	Dev CV (%)	Errors	Test (%)	95% CI	Missed	False al.	AUC	p vs LR
Logistic regression	0.56	1/270	0.37	0.1–2.1	0	1	0.9999	–
kNN	0.09	1/270	0.37	0.1–2.1	0	1	0.9966	1.00
Random forest†	0.56	3/270	1.11	0.4–3.2	0	3	0.9998	0.62
Gradient boosting†	0.28	1/270	0.37	0.1–2.1	0	1	0.9999	1.00
Neural network†	0.00	0/270	0.00	0.0–1.4	0	0	1.0000	1.00

Notes needed to reach a target. Table 2 turns the curves into the quantity a practitioner would ask for. To reach 2% error, the neural network needs about 50 notes and logistic regression 71, with kNN close behind at 85. To reach 1%, the order changes: the neural network needs about 92, kNN 149, and logistic regression 345, more than twice as many as kNN. The forest needs the most at both targets. The bootstrap intervals for the core methods do not overlap at 1%.

5.3 Several images per specimen

If kNN with $k = 1$ wins only because it finds a near-copy of the same physical note in the training set, its advantage should shrink when whole pseudo-specimens are kept on one side of the split. It does not (Figure 2, Table 9). With grouped draws, kNN errs on 0.74% at 200 notes and 0.13% at 800, against 0.81% and 0.13% with random draws; logistic regression moves from 0.65% to 0.71% at 800. Grouped 5-fold CV on all development notes gives the same picture (Section 3; logistic regression 0.65% and kNN 0.09% in both cases). Within the limits of our heuristic grouping, the crossover is not an artefact of specimen leakage.

5.4 Held-out test set

Table 3 gives the result of the single test evaluation. Logistic regression, kNN and boosting each misclassify 1 of 270 notes (0.37%), the forest 3, and the neural network none. Every error is a false alarm; no method misses a forgery. These counts are too small to rank the methods: the 95% intervals overlap heavily, and no McNemar test against the baseline comes close to significance ($p \geq 0.62$). This is the outcome we expected before running it, and it is why the learning curves, not the test set, carry our conclusions. ROC curves are in Appendix A.5.

6 Discussion

Why the curves look the way they do. We think the ranking follows the shape of decision boundary each method can represent. The evidence points to a smooth, curved boundary: no plane separates the development notes, but the 8-unit network separates all of them (0.00% CV error on the full development set, Table 3). Logistic regression fits a plane with five parameters, so it is reliable from few notes but cannot get below its bias floor of about 0.6%. kNN with $k = 1$ can follow any boundary, but it needs enough notes to fill the space near it, so it starts badly and improves steadily. Trees split on one feature at a time, so they approximate a smooth oblique boundary with many small steps, and that needs many notes. The small neural network sits in between: with 8 hidden units (49 weights) it can represent a smooth curved boundary, yet it has far fewer degrees of freedom than kNN or a forest. On this dataset that appears to be the right inductive bias at every sample size we tried, although we did not test this explanation directly.

What a practitioner should take away. If only a few dozen labelled notes are available, logistic regression or a small neural network is the safe choice; kNN and tree ensembles should not be used below about 100 notes. With a few hundred notes, the gap between the methods is a fraction of a percentage point, and the choice can be made on other grounds, such as interpretability (logistic regression) or simplicity (kNN). The answer to "which method is best?" depends on n, which a single accuracy number hides.

Limitations.

- **One dataset, four features.** The conclusions are about this benchmark. They are not a claim about real currency authentication, where the notes, scanners and forgery methods vary far more.
- **Specimen identity is unknown.** Our 8-row blocks are a heuristic inferred from file order. They may cut real specimens in two or join different ones, so the grouped check reduces the risk of leakage but cannot rule it out. The dev/test split itself was random, so test notes may share specimens with development notes.
- **The SEs understate uncertainty.** All draws come from the same 1,078 notes, and at large n they share most of their training and evaluation notes. The SEs measure variation across subsamples of this dataset, not across new datasets, and the bootstrap intervals in Table 2 inherit this.
- **Small grids and one initialisation.** Each method was tuned on a small grid fixed before the experiments, and each network was trained from one random initialisation. A wider search could move individual curves, though the larger forest grid did not.
- **A small test set.** With 270 notes, one error is 0.37 percentage points. The test set confirms that no method collapses on new data, but it cannot rank them.
- **Label coding.** We assume class 1 is forged. If it is the other way round, "missed forgeries" and "false alarms" swap; errors and AUC do not change.

7 Conclusion

On the UCI banknote data, the question "which classifier is most accurate?" has no useful answer: every method we tried makes at most 3 errors on 270 test notes. The question "how many labelled notes does each method need?" does. Logistic regression reaches 2% error fastest of the two core methods but levels off near 0.6%. kNN overtakes it at about 100 notes and needs less than half as many notes to reach 1%. Of the later-course methods, tree ensembles are the slowest learners, while a small neural network is the most sample-efficient method at every size. These conclusions survive a check for several images of the same specimen. More generally, when a benchmark is saturated, learning curves with paired comparisons are a cheap and honest way to still compare methods.

Acknowledgments and Disclosure of Funding

Contributions. The work was split evenly. Maanas built the data pipeline (loading, de-duplication and the fixed split), the logistic-regression baseline, kNN, the learning-curve framework and the

specimen-leakage check, and drafted the Data, Method and Results sections. Advit built the random-forest, gradient-boosting and neural-network extensions, the bootstrap analysis of notes needed, the test-set evaluation and the figures, and drafted the Introduction, Related work and Discussion. Both checked all code and numbers and edited the whole paper.

AI use. We used an LLM assistant (Claude, Anthropic) for code generation, debugging, literature search and drafting and copy-editing of the manuscript. All experimental results were produced and verified by the authors, and every reference was checked against its source.

Data and code. UCI Banknote Authentication, V. Lohweg, CC BY 4.0. Code, split and results: https://github.com/maanasgoel/ese5420-banknote.

References

[1] V. Lohweg. Banknote Authentication. UCI Machine Learning Repository, 2012. https://archive.ics.uci.edu/dataset/267/banknote+authentication. CC BY 4.0.

[2] E. Gillich and V. Lohweg. Banknotenauthentifizierung. In *1. Jahreskolloquium "Bildverarbeitung in der Automation"*, Lemgo, Germany, 2010.

[3] C. Perlich, F. Provost, and J. S. Simonoff. Tree induction vs. logistic regression: A learning-curve analysis. *Journal of Machine Learning Research*, 4:211–255, 2003.

[4] A. Y. Ng and M. I. Jordan. On discriminative vs. generative classifiers: A comparison of logistic regression and naive Bayes. In *Advances in Neural Information Processing Systems 14*, 2002.

[5] T. Viering and M. Loog. The shape of learning curves: A review. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(6):7799–7819, 2023.

[6] S. Varma and R. Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006.

[7] G. C. Cawley and N. L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.

[8] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):15, 2012.

[9] T. G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.

[10] G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning*. Springer, 2nd edition, 2021.

[11] T. Cover and P. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967.

[12] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[13] J. H. Friedman. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[14] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

A Additional results

A.1 Baseline details

Table 4 gives the baseline estimates on the development notes and Figure 3 the CV error across the C grid. Errors are flat from $C = 100$ to $C = 1000$, so the baseline is effectively unregularised. The first rows of the cleaned data are in Table 5.

Table 4: Logistic-regression baseline on the 1,078 development notes. SE is the standard error of the five fold errors. The 95% interval is a Wilson interval on the pooled out-of-fold errors.

Model / estimate	Error	SE	Notes
Majority class	45.3%	–	–
Logistic regression, $C = 100$ (5-fold CV)	0.56%	0.09%	6/1078; 95% CI 0.26%–1.21%
Logistic regression, nested 5×5 CV	0.74%	0.32%	AUC 0.9999
Logistic regression, 10 repeats of 5-fold	0.54%	–	range 0.37%–0.65%

Table 5: First rows of the cleaned data, two per class.

Variance	Skewness	Kurtosis	Entropy	Class
3.622	8.666	-2.807	-0.447	0
4.546	8.167	-2.459	-1.462	0
-1.397	3.319	-1.393	-1.995	1
0.390	-0.143	-0.032	0.351	1

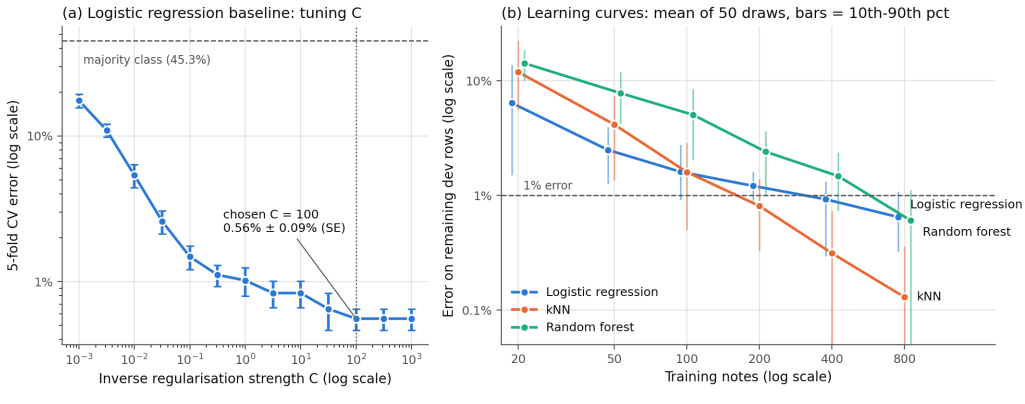


Figure 3: Baseline-checkpoint figure. (a) Logistic-regression CV error across the C grid, ± 1 SE. (b) The first learning curves (three methods, six sizes); the main text extends these.

A.2 Full learning curves

Table 6: Mean error (%) ± SE over 50 draws, scored on the development notes not used for training. Bold marks the lowest mean at each size. †Extension.

n	Logistic regression	kNN	Random forest†	Gradient boosting†	Neural network†
10	16.28 ± 1.51	21.57 ± 1.31	23.24 ± 1.47	21.00 ± 1.45	**11.90 ± 1.41**
20	6.39 ± 0.85	11.89 ± 1.04	14.22 ± 0.71	15.68 ± 0.78	**4.74 ± 0.51**
30	3.38 ± 0.33	7.11 ± 0.47	9.95 ± 0.52	11.70 ± 0.64	**2.90 ± 0.34**
50	2.50 ± 0.20	4.15 ± 0.35	7.81 ± 0.45	8.92 ± 0.51	**2.02 ± 0.20**
100	1.61 ± 0.11	1.60 ± 0.15	5.05 ± 0.34	5.34 ± 0.39	**0.91 ± 0.10**
150	1.22 ± 0.07	0.99 ± 0.10	3.56 ± 0.23	3.13 ± 0.19	**0.63 ± 0.08**
200	1.21 ± 0.06	0.81 ± 0.07	2.41 ± 0.15	1.91 ± 0.12	**0.49 ± 0.06**
300	1.07 ± 0.06	0.56 ± 0.06	1.99 ± 0.12	1.88 ± 0.14	**0.28 ± 0.05**
400	0.93 ± 0.05	0.31 ± 0.04	1.47 ± 0.09	1.17 ± 0.08	**0.12 ± 0.03**
600	0.82 ± 0.06	0.13 ± 0.03	1.05 ± 0.07	0.66 ± 0.07	**0.06 ± 0.02**
800	0.65 ± 0.06	0.13 ± 0.03	0.60 ± 0.06	0.33 ± 0.06	**0.00 ± 0.00**

Table 7: Paired difference from logistic regression, in percentage points (negative is better than the baseline), with the number of draws (out of 50) in which the method beat / lost to the baseline; the rest are ties. Bold: $|\Delta| > 2$ SE.

n	10	20	50	100	200	400	800
kNN	**+5.28** (16/34)	**+5.50** (9/40)	**+1.66** (14/35)	−0.01 (27/19)	**−0.40** (37/9)	**−0.61** (45/0)	**−0.52** (41/2)
Random forest†	**+6.96** (13/37)	**+7.83** (5/45)	**+5.31** (1/49)	**+3.44** (1/49)	**+1.19** (8/41)	**+0.55** (7/37)	−0.04 (21/20)
Gradient boosting†	**+4.72** (18/32)	**+9.29** (6/44)	**+6.43** (1/49)	**+3.74** (0/50)	**+0.69** (12/34)	**+0.24** (15/30)	**−0.32** (34/9)
Neural network†	**−4.39** (33/16)	**−1.65** (29/18)	**−0.47** (31/14)	**−0.70** (44/2)	**−0.72** (47/1)	**−0.81** (50/0)	**−0.65** (45/0)

Table 8: Most common hyperparameters chosen by inner CV, with the share of the 50 draws that chose them. For the forest, m is the number of features tried per split.

Method	$n = 10$	$n = 50$	$n = 200$	$n = 800$
Logistic regression	$C = 0.001$ (38%)	$C = 3.16$ (30%)	$C = 1$ (26%)	$C = 100$ (42%)
kNN	$k = 1$ (80%)	$k = 1$ (68%)	$k = 1$ (80%)	$k = 1$ (98%)
Random forest†	$m = 1$, leaf 1 (60%)	$m = 1$, leaf 1 (56%)	$m = 1$, leaf 1 (66%)	$m = 1$, leaf 1 (52%)
Gradient boosting†	$\eta = 0.05$, depth 1 (84%)	$\eta = 0.05$, depth 1 (52%)	$\eta = 0.2$, depth 1 (50%)	$\eta = 0.2$, depth 3 (86%)
Neural network†	8 units, $\alpha = 0.0001$ (54%)	8 units, $\alpha = 0.0001$ (50%)	8 units, $\alpha = 0.0001$ (44%)	8 units, $\alpha = 0.0001$ (72%)

A.3 Grouped draws

Table 9: Mean error (%) ± SE for random draws and for draws that keep each 8-row pseudo-specimen on one side of the split (training, inner CV folds and evaluation).

	Logistic regression		kNN	
n	random	grouped	random	grouped
50	2.50 ± 0.20	2.30 ± 0.28	4.15 ± 0.35	4.53 ± 0.44
100	1.61 ± 0.11	1.86 ± 0.17	1.60 ± 0.15	1.57 ± 0.18
200	1.21 ± 0.06	1.14 ± 0.04	0.81 ± 0.07	0.74 ± 0.06
400	0.93 ± 0.05	0.96 ± 0.06	0.31 ± 0.04	0.30 ± 0.05
800	0.65 ± 0.06	0.71 ± 0.08	0.13 ± 0.03	0.13 ± 0.03

A.4 Random-forest tuning

Table 10: Random forest on all development notes. Top: 5-fold CV error of the small grid used inside every draw against a 64-setting grid. Bottom: out-of-bag error against the number of trees ($m = 2$).

Grid	Configs	CV error (%)	Best setting
small (used in curves)	6	0.56 ± 0.27	$m = 2$, leaf 1, 200 trees
large	64	0.46 ± 0.21	$m = 3$, leaf 1, 500 trees, gini

Trees	25	50	100	200	500	1000
OOB error (%)	0.74	0.56	0.46	0.56	0.65	0.74

A.5 Test-set ROC curves

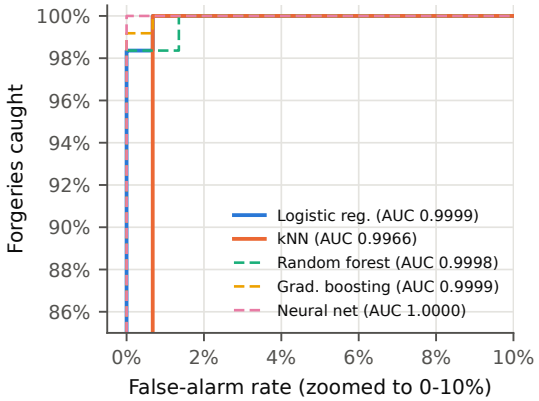


Figure 4: ROC curves on the 270 test notes, zoomed to the top-left corner.

B Reproducibility

All code, the fixed split and the result files are at https://github.com/maanasgoel/ese5420-banknote. `src/baseline.py`, `src/learning_curves.py`, `src/analyse.py` and `src/test_eval.py` produce every number, and `src/make_tables.py` writes every table and every number quoted in the text, so the paper cannot drift from the results. The full run takes about 20 minutes on an 8-core laptop (Python 3.10, scikit-learn 1.7.2).